

Dunes Insolites

Production Readiness Plan

Next.js public site & Spring Boot API — from current state to launch

Current status: NOT deployable. Three live security defects (one of them a remote privilege escalation to realm ADMIN), a CORS layer that is configured but never applied, and a frontend that shares *no endpoint* with the backend it is supposed to call.

This plan sequences 27 fixes across 6 phases. Phases 0–1 are mandatory before any public exposure. Phase 2 is the largest body of work and decides how much of the rest gets rewritten.

Prepared for	Abdou Hanafi
Date	25 August 2026
Frontend	Next.js 16.3.1 · React 19 · Tailwind v4 · ~6,000 LOC
Backend	Spring Boot 4.0.3 · Java 21 · Postgres · Keycloak · RabbitMQ · ~12,500 LOC
Defects tracked	27 (3 critical · 9 high · 11 medium · 4 low)
Estimated effort	24–34 working days

1. Executive summary

Both codebases are, in isolation, better than average. The frontend is strictly typed with zero `any` escapes, server-first, and routes all backend access through a single documented seam. The backend is cleanly layered with DTOs, MapStruct mappers, and a rich domain model. Neither is a rewrite candidate.

What blocks production is not code quality. It is three specific things:

1. **Live exploitable defects.** Anonymous registration accepts a caller-supplied `role`, so anyone can mint themselves an ADMIN account. Production secrets (Gmail app password, Keycloak client secret) are committed to git. Five controllers that perform destructive operations have no authorization rule beyond "is logged in".
2. **A non-functional integration layer.** CORS is written but never wired into the security filter chain, so every browser cross-origin request is rejected today.
3. **A contract gap.** The frontend calls `/activities`, `/stays`, `/bookings`, `/gallery`, `/stats`. The backend serves `/api/tours`, `/api/reservations`, `/api/extras`. There is no overlap. Worse, the frontend silently falls back to seed data on any fetch failure — so a misconfigured launch would serve fake prices indefinitely with no error anywhere.

Risk snapshot

RISK	SEV	CONSEQUENCE IF SHIPPED AS-IS
Role escalation via public registration	CRITICAL	Full administrative takeover by any anonymous internet user. Total data compromise.
Secrets committed to git	CRITICAL	Mailbox takeover; Keycloak client impersonation; token forgery.
CORS never applied to filter chain	CRITICAL	Every browser call fails. Site appears to work (seed fallback) while being entirely disconnected.
Unprotected destructive endpoints	HIGH	Any customer account can delete guides, drivers and booking sources.
Frontend/backend contract mismatch	HIGH	No real data ever reaches the site.
Silent fetch-failure fallback	HIGH	Stale marketing prices presented as live. Bookings taken at wrong amounts.
<code>ddl-auto: update</code> on remote Postgres	HIGH	Uncontrolled schema mutation; potential silent data loss on deploy.
In-memory booking store	HIGH	Bookings vanish on restart; inconsistent across serverless instances.
All RuntimeExceptions mapped to HTTP 400	HIGH	Internal errors leak messages; clients cannot distinguish auth failure from bad input.
Effectively zero test coverage	MEDIUM	No regression safety on a money-handling booking flow.

Phase plan at a glance

#	PHASE	DURATION	TASKS	EXIT GATE
0	Stop the bleeding	1 day	4	No remotely exploitable defect remains
1	Make the API usable	3–4 days	6	A browser can call the API and get correct status codes
2	Unify the contract	8–11 days	6	Site renders 100% live backend data, zero seed fallbacks
3	Data & correctness	5–7 days	5	Migrations versioned; booking flow covered by tests

4	Production hardening	5–7 days	4	Observability, rate limiting, TLS, CI green
5	Launch	2–4 days	2	Go/no-go checklist fully ticked
Total		24–34 days	27	

Estimates assume one developer working the plan sequentially. Phase 2 can partially parallelise if a second person takes the frontend adaptation while the first builds the missing backend endpoints.

0 Stop the bleeding

Day 1 · 4 tasks · blocking

Every item here is a live, remotely exploitable defect. None of them takes long. Do all four before writing another line of feature code, and before the API is reachable from the public internet.

SEC-01 Remove caller-controlled `role` from public registration

2h

KeycloakUserService.java:67 · RegisterRequest.java:11 · SecurityConfig.java:45

Why: `POST /api/auth/register` is `permitAll()` and passes the request body's `role` straight into `assignRole()`. An anonymous `{"role":"ADMIN"}` yields a Keycloak realm ADMIN account. This is a complete authentication bypass.

Do:

- Delete the `role` field from `RegisterRequest` entirely — do not merely validate it.
- Hard-code the self-registration role in `registerUser()`.
- If partner self-signup is a real requirement, accept a separate `accountType` restricted to CLIENT/PARTENAIRE, and gate PARTENAIRE behind manual admin approval rather than instant role assignment.
- Confirm `POST /api/users/add` (the admin path, already `hasRole('ADMIN')`) remains the *only* way to create ADMIN or CAMPING users.

```
// KeycloakUserService.registerUser()
- assignRole(keycloakUserId, request.getRole().name());
+ assignRole(keycloakUserId, UserRole.CLIENT.name());
```

Done when: a crafted registration request containing `"role":"ADMIN"` produces a CLIENT account, verified by decoding the returned token's `realm_access.roles`.

SEC-02 Rotate and un-commit all secrets

3h

backend/duneinsolite/.env · .gitignore

Why: `.env` is tracked in git — while its own first line claims it is gitignored. It is not. The Gmail app password and the Keycloak client secret are in history.

Do, in this order:

1. **Rotate first, clean second.** Revoke the Gmail app password in the Google account; generate a new Keycloak client secret for `duneinsolite-api`. Anything already committed must be treated as public.
2. Add `.env` to `.gitignore`; run `git rm --cached .env`.
3. If the repo was ever pushed to a shared or public remote, scrub history with `git filter-repo` and force-push — then rotate again, since the window was open.
4. Move production values into the deployment platform's secret store (systemd `EnvironmentFile` with `chmod 600`, Docker secrets, or the host's vault). Keep `.env.example` with placeholder values only.
5. Remove the `SPRING_MAIL_USERNAME` default `bacemakkari25@gmail.com` from `application.yml` — a personal address should not be a shipped fallback.

Done when: `git ls-files | grep env` returns only `.env.example`, and the old credentials are confirmed rejected by their providers.

SEC-03 Apply CORS to the security filter chain

1h

[SecurityConfig.java](#) · [CorsConfig.java](#)

Why: `CorsConfig` declares a `CorsConfigurationSource` bean, but `SecurityConfig` never calls `.cors(...)`, and there is no `CorsFilter` or `WebMvcConfigurer` anywhere in the tree. In Spring Security 6/7 the bean alone is inert. Every browser cross-origin request is currently rejected.

```
// SecurityConfig.securityFilterChain()
http
+   .cors(Customizer.withDefaults())
  .csrf(csrf -> csrf.disable())
  .sessionManagement(...)
```

While in the file, also fix the dead `AsyncSupportConfigurer` bean at `CorsConfig.java:44`. That class is a callback object Spring hands to `WebMvcConfigurer.configureAsyncSupport()`; constructing one as a `@Bean` configures nothing, so the SSE endpoint currently has no timeout override. Delete the bean and set the property instead:

```
spring:
  mvc:
    async:
      request-timeout: -1 # SSE streams must not time out
```

Done when: a preflight `OPTIONS` from an allowed origin returns 200 with `Access-Control-Allow-Origin`, and a disallowed origin is refused.

SEC-04 Authorize the five open controllers

3h

[ChauffeurController](#) · [GuideController](#) · [SourceController](#) · [NotificationController](#) · [CampingSettingsController](#)

Why: these carry no `@PreAuthorize` and have no matching rule in `SecurityConfig`, so they fall through to `anyRequest().authenticated()`. Any logged-in CLIENT can `DELETE /api/guides/{id}`, `DELETE /api/sources/{id}` or `PATCH /api/chauffeurs/{id}`.

Do:

- Staff and reference data (`guides`, `chauffeurs`, `sources`): `hasAnyRole('ADMIN', 'CAMPING')` for reads, `hasRole('ADMIN')` for mutations.
- `notifications`: authenticated, but every handler must scope to the calling principal — a user must not be able to read or mark-read another user's notifications by guessing an id.
- Audit the whole chain for the same class of gap: any controller method without an explicit rule is implicitly "any logged-in user".

Done when: an integration test authenticating as CLIENT receives 403 on every staff-mutation endpoint.

PHASE 0 EXIT GATE

- ☐ Registration cannot yield any role above CLIENT, verified by request, not by reading code
- ☐ All leaked credentials rotated at the provider; `.env` untracked
- ☐ A real browser can complete a cross-origin request against the API
- ☐ CLIENT-authenticated calls to staff endpoints return 403

Do not proceed past this gate with any item unticked.

1 Make the API usable

Days 2–5 · 6 tasks

The API is now safe but still hostile to a client: wrong status codes, unvalidated input, entity leakage. This phase makes it behave predictably before anything is built on top of it.

API-01 Replace the blanket `RuntimeException` → 400 mapping

1d

`GlobalExceptionHandler.java:61` · 37 throw sites across services

Why: every `RuntimeException` becomes `400 Bad Request` carrying `ex.getMessage()`. A failed login returns 400 rather than 401. A genuine NPE, a Keycloak outage or a dead database connection all surface as "400" with an internal message echoed to the caller — misleading and an information leak.

Do:

- Introduce a `BusinessException` base type; map *that* to 400. Let unmatched exceptions fall to the existing generic 500 handler with its safe message.
- Sweep all 37 `throw new RuntimeException(...)` sites and reclassify each: business rule (400), not found (404), conflict (409), auth failure (401), forbidden (403).
- `AuthService.login()` currently catches *everything* and rethrows `RuntimeException("Invalid credentials")`, so a Keycloak outage is reported to the user as a wrong password. Separate transport failure (503) from a genuine 401 from Keycloak.
- Never place `ex.getMessage()` of an unclassified exception into a response body.

Done when: bad password → 401; unknown reservation id → 404; duplicate email → 409; database down → 500 with a generic message and a full server-side log.

API-02 Enable bean validation everywhere

4h

`AuthController`, `ChauffeurController`, `GuideController`, `NotificationController`, `ProduitController`, `StatisticsController`, `CurrencyController`

Why: 7 of 19 controllers never annotate `@RequestBody` with `@Valid`, so any constraint annotations on their DTOs are silently ignored. `AuthController` is among them — registration, login and refresh accept anything, including null passwords.

Do: add `@Valid` to every request body; add real constraints to `RegisterRequest` and `LoginRequest` (`@NotBlank`, `@Email`, password length and complexity); confirm `MethodArgumentNotValidException` still renders the structured field-error body the frontend expects.

Done when: an empty registration body returns 400 with a per-field error map, not a 500.

API-03 Make Keycloak/DB registration atomic

4h

`KeycloakUserSyncService.java:59–71`

Why: the Keycloak account is created *and a role assigned* before the local `existsByEmail` check throws. `@Transactional` rolls back Postgres but has no authority over Keycloak, so every duplicate registration leaves an orphaned, role-bearing Keycloak user behind.

Do: move the `existsByEmail` check to the top of the method, before any remote call; wrap the Keycloak creation so that any subsequent failure triggers a compensating `users().delete()`; log orphans that cannot be cleaned so they can be reconciled. Apply the same ordering to the admin-created user path.

Done when: registering an existing email twice leaves exactly one Keycloak user and returns 409.

API-04 Stop returning JPA entities from controllers

2h

`AuthController.java:38`

Why: `register()` returns the raw `User` entity. That leaks internal columns, couples the wire format to the schema, and risks lazy-initialisation serialization errors outside a transaction.

Do: return the existing `UserResponse` DTO via `UserMapper`. Sweep other controllers for the same pattern.

Done when: no controller signature in the project references a `model.*` type.

API-05 Decide and document the session model

1d

`lib/api.ts:214` · `LoginResponse.java` · `AuthController`

Why: the frontend explicitly asks for an `httpOnly/Secure/SameSite` cookie and argues — correctly — that a token parked in `localStorage` is readable by any injected script. The backend returns `accessToken` + `refreshToken` in the JSON body. These are incompatible and the decision blocks Phase 2.

Recommended: the BFF pattern. It satisfies both sides without changing the Angular admin apps that already consume bearer tokens:

- The Next.js route handlers under `app/api/auth/*` proxy to Spring, receive the token pair, and set them as `httpOnly` cookies on the browser response.
- Server components and route handlers read the cookie and attach `Authorization: Bearer` when calling Spring. The token never reaches client-side JavaScript.
- Spring's contract is unchanged, so the `admin`, `partner` and `camping` apps keep working as they do now.
- Refresh happens server-side in the Next layer against `POST /api/auth/refresh`.

Consequence: once auth flows through Next route handlers rather than the browser, `NEXT_PUBLIC_API_URL` is no longer the right switch for authenticated calls — see CON-01.

Done when: the decision is written into `API_CONTRACT.md` and a login sets an `httpOnly` cookie with no token visible in `document.cookie` or `localStorage`.

API-06 Restore the CSRF story lost with cookies

4h

`SecurityConfig.java:40`

Why: `csrf.disable()` is defensible for a purely bearer-token, stateless API. The moment API-05 introduces cookie-borne credentials on the public site, that assumption no longer holds for the Next layer.

Do: keep Spring stateless and CSRF-disabled — it still only accepts bearer headers — and enforce CSRF at the Next.js BFF boundary instead: `SameSite=Lax` on the session cookie plus an origin check in the route handlers that perform mutations.

Done when: a cross-site form POST to a Next mutation route is rejected.

PHASE 1 EXIT GATE

- ☐ Status codes are semantically correct across auth, not-found, conflict and server-error paths
- ☐ No internal exception message is reachable by a client
- ☐ All request bodies validated; empty or malformed input yields structured 400s
- ☐ Session model decided, documented, and implemented end to end

2 Unify the contract

Days 6–16 · 6 tasks · largest phase

This is the structural work. Today the two systems model different domains and share no endpoint. The frontend's own `API_CONTRACT.md` documents a shape the backend does not implement, and the backend's CORS list targets an entirely different product — the Angular admin suite on ports 4200–4202.

The gap, precisely

FRONTEND CALLS	BACKEND OFFERS	RESOLUTION
<code>/activities</code>	<code>/api/tours</code> , <code>/api/tour-types</code>	Map — add public projection
<code>/stays</code>	<i>nothing</i>	Build — model does not exist
<code>/bookings</code> , <code>/availability</code>	<code>/api/reservations</code> (different shape)	Map, plus build availability
<code>/reviews</code> (public)	<code>/api/reviews</code> (<code>isAuthenticated()</code>)	Add public read projection
<code>/gallery</code>	<i>nothing</i>	Build or keep static
<code>/stats</code>	<i>nothing</i> (admin stats exist, wrong shape)	Build public counters
<code>/contact</code> , <code>/subscribe</code>	<i>nothing</i>	Build
<code>/auth/login</code> → cookie	<code>/api/auth/login</code> → JSON tokens	Resolved in API-05

CON-01 Settle the domain vocabulary — one decision, everything follows

1d

Why: `Tour` vs `Activity` , `Reservation` vs `Booking` , and the entirely absent `Stay` . Every subsequent task depends on this answer, so make it explicitly and write it down.

Recommendation: keep the backend's vocabulary as canonical, translate at the edge. The backend model is richer, already in production use by the admin suite, and renaming it would break three deployed Angular apps. Add a thin public-facing controller layer that projects `Tour` → `Activity` JSON in exactly the shape `API_CONTRACT.md` specifies. The frontend does not change at all.

- New package `controller.publicapi` serving `/api/public/**` : all `permitAll()` , all read-only, all cacheable.
- Dedicated response DTOs — do not reuse the admin DTOs, whose fields (cost prices, commission rates, partner data) must never reach a public page.
- Set `NEXT_PUBLIC_API_URL` to `https://api.../api/public` ; authenticated traffic goes through the Next BFF instead.
- **Open question to resolve now:** where does `Stay` (nuitée) live? If a stay is a `TourType` with accommodation, say so explicitly; if it is a new entity, it needs a migration. `ReservationTourHebergement` suggests the concept partly exists already — confirm before building.

Done when: `API_CONTRACT.md` is updated with the agreed mapping table and signed off by both sides.

CON-02 Build the public read endpoints

3d

Implement `/api/public/activities`, `/stays`, `/reviews`, `/gallery` and `/stats` against the shapes in `API_CONTRACT.md`. Notes per endpoint:

- **activities** — project from `Tour / TourType`. `slug` does not exist on the entity today and must be added as a unique column, because it is the public route key.
- **reviews** — public, unauthenticated, filterable by `?activity=` and `?stay=`. Must return only approved reviews; add a moderation flag if absent. Preserve the guest's original language, per the contract.
- **gallery and stats** — low churn. If there is no CMS need, serve them from configuration rather than building tables for them.
- Honour the frontend's caching intent: `revalidate` is 300s for activities and stays, 600s for gallery and reviews, 3600s for stats. Set matching `Cache-Control` headers.
- Exclude every internal field — cost, margin, commission, partner identity, staff assignment.

Done when: each endpoint's response validates against the JSON examples in `API_CONTRACT.md`, including exact enum casing (`golden-hour`, `Easy`).

CON-03 Build real availability

2d

replaces `lib/bookings.ts:44 baselineTaken()`

Why: the frontend currently computes availability from a *hash of the date string* — deterministic fake data, explicitly labelled as such. Real capacity lives in `ReservationCapacityValidator` on the backend.

Do: expose `GET /api/public/availability?activity={slug}&date={iso}` returning the contract's `{ slots: [{ slot, seatsLeft, available }] }`, computed from real reservations and the tour's capacity. Reuse `ReservationCapacityValidator` rather than reimplementing the rule. Guard against the obvious race: availability shown is advisory, and capacity must be re-checked inside the booking transaction.

Done when: creating a booking visibly decrements `seatsLeft` on the next availability call, and an over-capacity booking is rejected server-side even if the UI allowed it.

CON-04 Wire public booking to real reservations

3d

lib/bookings.ts · app/api/bookings/route.ts · ReservationServiceImpl

Why: lib/bookings.ts stores bookings in an in-memory Map that dies on restart and is per-instance on serverless. The TODO: persist to the database and send the confirmation email at app/api/bookings/route.ts:26 is the whole feature.

Do:

- Decide whether a public booking creates a guest User or an anonymous reservation. Guest checkout without forced account creation is strongly preferable for conversion; the reservation still needs an owner record for the invoice.
- Map BookingInput → ReservationRequest, calling the existing service so pricing, capacity validation and invoice generation are not duplicated.
- Server-side pricing is authoritative. The frontend computes total = priceFrom × partySize for display only — never trust a client-sent total.
- Return the contract's Booking shape including the human-readable DI-XXXXXX reference.
- Hook the existing EmailService for confirmation mail; make it asynchronous via the RabbitMQ path already in place so a slow SMTP does not block the booking response.
- GET /api/public/bookings/{id} for the confirmation page must be unguessable — the 6-character DI- reference is too weak on its own. Require reference plus email, or issue an opaque lookup token.

Done when: a booking made on the site appears in the admin app, survives a backend restart, and triggers a confirmation email.

CON-05 Build contact and newsletter endpoints

1d

Both are unauthenticated write endpoints on a public site, so they are the primary spam and abuse surface. Rate-limit per IP, add a honeypot field or captcha, validate and normalise the email, and store subscriptions with a double opt-in token rather than mailing immediately. Route contact submissions through the existing EmailService.

Done when: both endpoints persist and notify, and a scripted flood is throttled.

CON-06 Make seed-data fallback loud, then remove it

1d

lib/api.ts:68-77

Why: this is the most dangerous line in the frontend. get() catches every failure and returns seed data. In production a dead backend renders stale marketing prices as though they were live — no log, no alert, no visible signal. Bookings could be taken at the wrong price for days before anyone noticed.

Do:

- When usingRemoteApi is true, log every fallback server-side at error level with the path and status.
- Distinguish the two intents: seed fallback is a development convenience, not a production resilience strategy. Gate it on NODE_ENV !== "production".
- In production, prefer stale-but-real cached data, and an honest error state over invented content, for anything price-bearing.
- Add a startup assertion: if NODE_ENV === "production" and NEXT_PUBLIC_API_URL is unset, fail the build rather than shipping a seed-data site.

Done when: stopping the backend produces visible errors in logs and an honest UI state, not silent fake prices.

PHASE 2 EXIT GATE

- ☐ Every page renders live backend data with `NEXT_PUBLIC_API_URL` set
- ☐ Zero seed-data fallbacks occur during a full click-through of the site
- ☐ A booking made on the public site is visible in the admin app
- ☐ `API_CONTRACT.md` matches what the backend actually serves

DAT-01 Replace `ddl-auto: update` with versioned migrations

2d

`application.yml`

Why: Hibernate is currently free to mutate a live remote Postgres (`79.143.185.33:5433`) on every boot. `update` never drops or narrows safely, silently diverges between environments, and gives no rollback path. It is the single largest data-loss risk in the stack.

Do: adopt Flyway. Baseline the current schema from the existing database, then take every future change as a numbered migration. Set `ddl-auto: validate` in all environments so the app refuses to start against a schema it does not expect. Add `show-sql: false` outside development — it is currently `true` and will flood production logs.

Done when: a fresh database can be built from migrations alone, and the app fails fast on schema drift.

DAT-02 Test the booking path

3d

Why: ~12,500 backend LOC and ~6,000 frontend LOC are covered by exactly one test — the default empty context-load. The frontend has no test script at all. This is a money-handling flow.

Priority order — do not chase a coverage percentage, cover these paths:

1. Authorization matrix: for each role, assert allowed and forbidden on every endpoint. This is the regression net for SEC-01 and SEC-04 and is the highest-value suite in the project.
2. Pricing: total calculation, discounts (`UserProductRemise`), currency conversion.
3. Capacity: `ReservationCapacityValidator`, including the concurrent over-booking case.
4. Reservation status transitions and the recently-fixed cancellation NPE path.
5. Frontend: `validateBooking` unit tests, plus one end-to-end booking happy path.

Use Testcontainers for Postgres so tests run against the real engine. Add a `test` script to the frontend `package.json` — there is none today.

Done when: the authorization matrix and pricing suites pass in CI, and deliberately reintroducing the SEC-01 defect makes a test fail.

DAT-03 Bound the unbounded queries

1d

`11 sites, notably ReservationServiceImpl.java:487`

`findAll()` followed by in-memory `.stream().filter().map()` appears in 11 service methods. Acceptable at seed scale; linear degradation as reservations accumulate, and `ReservationServiceImpl:487` loads the entire reservation table. Push filtering into the repository — `ReservationSpecification` already exists for exactly this — and paginate every list endpoint that can grow.

Done when: no service method loads a growth table in full, and list endpoints accept `page` and `size`.

DAT-04 Split `ReservationServiceImpl`

2d

1,788 lines in one class. Split along the seams already visible in the method names: creation, status transitions, staff assignment, pricing and repartition, querying. Do this *after* DAT-02 so the tests catch what the refactor breaks — not before.

Optional. Skip if the schedule is tight; it is maintainability, not correctness. But every future change to booking logic pays interest until it is done.

Done when: no class in the service layer exceeds roughly 400 lines and the test suite is still green.

Confirm every monetary field is `BigDecimal` with an explicit scale and rounding mode — never `double`. Check that `CurrencyConfig` conversion rounds consistently, that the rate source has a defined refresh and a stale-rate fallback, and that the currency in which a price was quoted is stored alongside the amount. Note that the frontend contract types `priceFrom` as `int` euros; confirm that truncation is intended and matches what the backend stores.

Done when: a booking's displayed price, stored price and invoice line all agree to the cent in every supported currency.

OPS-01 Configuration and environment separation

2d

- Remove the hardcoded `79.143.185.33` defaults from all five sites in `application.yml`. A production-shaped default that silently works is worse than a missing one that fails loudly — require the variable and fail startup if absent.
- Split profiles: `dev`, `staging`, `prod`. `show-sql` and verbose logging in dev only.
- Drive `CorsConfig`'s origin list from configuration, not source. It currently mixes real domains with `http://` variants of admin subdomains — drop every plaintext-HTTP origin for production.
- Terminate TLS in front of the API. Keycloak on plain `http://79.143.185.33:8180` means tokens and the admin client secret cross the network in clear text.
- Note the two-domain split in the CORS list — `dunesinsolites.com` and `dunes-insolites.com` are both present. Confirm which is canonical and redirect the other.

Done when: the app cannot start in prod without explicit configuration, and no plaintext HTTP origin is accepted.

OPS-02 Rate limiting and abuse protection

1d

Currently absent everywhere. Minimum viable set: login attempts per IP and per account with lockout and backoff (this protects both your API and Keycloak); registration per IP; contact and newsletter per IP; a global ceiling on public read endpoints. Bucket4j in front of the filter chain, or at the reverse proxy — either is fine, but it must exist before the API is public.

Done when: a scripted credential-stuffing attempt is throttled and logged.

OPS-03 Observability

2d

- Actuator currently exposes only `health`. Add `metrics` and `prometheus`, bound to an internal interface, not the public one.
- Structured JSON logging with a correlation id threaded from the Next BFF through to Spring, so one booking can be traced across both systems.
- Alert on: seed-data fallback in production (CON-06), 5xx rate, failed logins, RabbitMQ consumer failures after retry exhaustion, SMTP failures.
- Ensure no log line ever contains a token, password or card-adjacent data. `AuthService` currently logs the email on every failed login — acceptable, but review the rest.

Done when: a booking can be traced end to end from a single correlation id, and an induced backend outage fires an alert.

OPS-04 CI/CD pipeline

2d

No pipeline exists in either repo. Build one that, on every push: builds both projects, runs the Phase 3 test suites, runs `eslint` and `tsc --noEmit`, runs a dependency vulnerability scan, and runs a secret scanner — the last one would have caught SEC-02. Deploy to staging on merge, to production on tag, with a documented rollback.

Consider consolidating the two repositories, or at least pinning their versions to each other. Independent repos with a shared contract drift, and the contract is exactly what broke here.

Done when: a red test blocks a merge, and a committed secret blocks a push.

LAU-01 Pre-launch verification

2d

- Full click-through of the site against production configuration, watching logs for fallbacks.
- Penetration pass focused on what this codebase got wrong: role escalation via any body field, IDOR on `/bookings/{id}` and `/notifications`, the authorization matrix by role.
- Load test the booking endpoint, including the concurrent-capacity case.
- Restore drill: prove a database backup actually restores. Confirm backups exist at all — nothing in the repository indicates they do.
- Legal pages exist (`/legal/privacy`, `/legal/terms`) and `CookieConsent` is implemented — have both reviewed against GDPR given European guests, and confirm the consent gate actually blocks what it claims to.

LAU-02 Deferred to post-launch

Explicitly out of scope for v1, listed so they are decisions rather than oversights:

- **i18n.** `LanguageSwitcher` is built but `next-intl` is not wired; only English exists. For a Tunisian desert camp serving French, German and Italian guests this is a real revenue gap — but it is additive and does not block launch. Schedule it as the first post-launch epic.
- `getBooking` in `lib/api.ts:191` bypasses the shared fetch helper — no revalidate, inconsistent error handling. Tidy when next touched.
- 235 MB in `assets-source/` is mostly untracked video; only four images are in git. Fine for now — keep it that way.
- `DuneinsoliteApplicationTests` and the placeholder `pom.xml` metadata (`<description>Demo project for Spring Boot</description>`) should be cleaned up.

Go / no-go checklist

Security — all mandatory

- ☐ Registration cannot escalate role
- ☐ All leaked secrets rotated
- ☐ `.env` untracked; secret scanning in CI
- ☐ Authorization matrix tested per role
- ☐ No entity leakage from controllers
- ☐ Rate limiting on all public writes
- ☐ TLS everywhere, Keycloak included
- ☐ Booking lookup not guessable

Data — all mandatory

- ☐ Flyway migrations; `ddl-auto: validate`
- ☐ Backups verified by restore drill
- ☐ Money in `BigDecimal` end to end
- ☐ No unbounded queries on growth tables

Integration — all mandatory

- ☐ CORS verified from a real browser
- ☐ Zero seed fallbacks in production
- ☐ Build fails if API URL unset in prod
- ☐ Booking reaches the admin app
- ☐ Confirmation email delivered
- ☐ Availability reflects real capacity

Operations

- ☐ CI green; red tests block merge
- ☐ Alerting on 5xx, fallbacks, SMTP, MQ
- ☐ Correlation ids across both systems
- ☐ Rollback procedure documented and tried
- ☐ Privacy and terms reviewed for GDPR

NON-NEGOTIABLE

The four Phase 0 items and the security column above are launch blockers, not preferences. Everything else can be traded against schedule with an explicit decision; these cannot.

Appendix — defect register

ID	SEV	DEFECT	LOCATION	PHASE	EST.
SEC-01	CRIT	Role escalation via public registration	KeycloakUserSyncService:67	0	2h
SEC-02	CRIT	Secrets committed to git	.env (tracked)	0	3h
SEC-03	CRIT	CORS bean never applied to filter chain	SecurityConfig / CorsConfig	0	1h
SEC-04	HIGH	Five controllers with no authorization rule	Chauffeur, Guide, Source, Notification, CampingSettings	0	3h
API-01	HIGH	All RuntimeExceptions map to 400; message leakage	GlobalExceptionHandler:61 (+37 sites)	1	1d
API-02	HIGH	Bean validation inactive in 7 controllers	AuthController et al.	1	4h
API-03	HIGH	Non-atomic Keycloak/DB registration	KeycloakUserSyncService:59	1	4h
API-04	MED	JPA entity returned from controller	AuthController:38	1	2h
API-05	MED	Cookie vs bearer session model conflict	lib/api.ts:214 / LoginResponse	1	1d
API-06	MED	CSRF disabled once cookies are introduced	SecurityConfig:40	1	4h
CON-01	HIGH	Domain vocabulary mismatch; no shared endpoint	architecture	2	1d
CON-02	HIGH	Five public read endpoints do not exist	new controller.publicapi	2	3d
CON-03	HIGH	Availability is a hash of the date string	lib/bookings.ts:44	2	2d
CON-04	HIGH	Bookings held in an in-memory Map	lib/bookings.ts:8	2	3d
CON-05	MED	Contact and newsletter endpoints missing	backend	2	1d
CON-06	HIGH	Silent seed-data fallback masks outages	lib/api.ts:68	2	1d
DAT-01	HIGH	<code>ddl-auto: update</code> on live Postgres	application.yml	3	2d
DAT-02	MED	Effectively zero test coverage	both repos	3	3d
DAT-03	MED	11 unbounded <code>findAll()</code> queries	service layer	3	1d
DAT-04	LOW	1,788-line service class	ReservationServiceImpl	3	2d
DAT-05	MED	Money-handling verification	CurrencyConfig, invoicing	3	1d

OPS-01	HIGH	Hardcoded IPs, no profiles, plaintext Keycloak	application.yml, CorsConfig	4	2d
OPS-02	HIGH	No rate limiting anywhere	backend	4	1d
OPS-03	MED	Observability limited to /health	application.yml	4	2d
OPS-04	MED	No CI/CD, no secret scanning	both repos	4	2d
LAU-01	MED	Pre-launch verification and restore drill	—	5	2d
LAU-02	LOW	i18n stub, minor cleanups (deferred)	frontend	post	—

Three decisions needed before Phase 2 starts

1. **Vocabulary.** Confirm the backend stays canonical and the public API translates at the edge — or decide otherwise now, because everything in Phase 2 depends on it.
2. **Where does Stay live?** A `TourType` with accommodation, or a new entity needing a migration? `ReservationTourHebergement` suggests the concept partly exists.
3. **Guest checkout or forced account?** Decides whether public booking creates a `User`, and shapes both the auth flow and the conversion funnel.

Prepared 25 August 2026 from a static analysis of both repositories at `master` — frontend `d4c404e`, backend `ae970bc`. Effort estimates assume one developer working sequentially and exclude review and deployment time.